



Python 程序设计基础与应用

张浩

18522982679

QQ 849993762

1.1 Python语言简介



- Python由荷兰数学和计算机科学研究学会的吉多·范罗苏姆于1990年代初设计，作为一门叫做ABC语言的替代品。
- Python的创始人为荷兰人吉多·范罗苏姆（Guido van Rossum）。1989年圣诞节期间，在阿姆斯特丹，Guido为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，作为ABC语言的一种继承。之所以选中Python（大蟒蛇的意思）作为该编程语言的名字，是取自英国20世纪70年代首播的电视喜剧《蒙提·派森的飞行马戏团》（Monty Python's Flying Circus）。

1.1 Python语言简介



- **Python** 特点

- **1.易于学习:** Python有相对较少的关键字, 结构简单, 和一个明确定义的语法, 学习起来更加简单。
- **2.易于阅读:** Python代码定义的更清晰。
- **3.易于维护:** Python的成功在于它的源代码是相当容易维护的。
- **4.一个广泛的标准库:** Python的最大的优势之一是丰富的库, 跨平台的, 在UNIX, Windows和Macintosh兼容很好。
- **5.互动模式:** 互动模式的支持, 您可以从终端输入执行代码并获得结果的语言, 互动的测试和调试代码片断。
- **6.可移植:** 基于其开放源代码的特性, Python已经被移植 (也就是使其工作) 到许多平台。
- **7.可扩展:** 如果你需要一段运行很快的关键代码, 或者是想要编写一些不愿开放的算法, 你可以使用C或C++完成那部分程序, 然后从你的Python程序中调用。
- **8.数据库:** Python提供所有主要的商业数据库的接口。
- **9.GUI编程:** Python支持GUI可以创建和移植到许多系统调用。
- **10.可嵌入:** 你可以将Python嵌入到C/C++程序, 让你的程序的用户获得"脚本化"的能力。

1.1 Python语言简介



- Python 是一个高层次的结合了解释性、编译性、互动性和面向对象的脚本语言。
- Python 的设计具有很强的可读性，相比其他语言经常使用英文关键字，其他语言的一些标点符号，它比其他语言更有特色语法结构。
- **Python** 是一种解释型语言：这意味着开发过程中没有了编译这个环节。类似于PHP和Perl语言。
- **Python** 是交互式语言：这意味着，您可以在一个 Python 提示符 `>>>` 后直接执行代码。
- **Python** 是面向对象语言：这意味着Python支持面向对象的风格或代码封装在对象的编程技术。
- **Python** 是初学者的语言：Python 对初级程序员而言，是一种伟大的语言，它支持广泛的应用程序开发

1.1 Python语言简介

- **Python** 发展历史



- Python 本身也是由诸多其他语言发展而来的,这包括 ABC、Modula-3、C、C++、Algol-68、SmallTalk、Unix shell 和其他的脚本语言等等。
- 像 Perl 语言一样, Python 源代码同样遵循 GPL(GNU General Public License)协议。
- 现在 Python 是由一个核心开发团队在维护, Guido van Rossum 仍然占据着至关重要的作用, 指导其进展。
- Python 2.7 被确定为最后一个 Python 2.x 版本, 它除了支持 Python 2.x 语法外, 还支持部分 Python 3.1 语法。

1.1 Python语言简介

- Python是一门跨平台、开源、免费的解释型高级动态编程语言。
- Python支持命令式编程（How to do）、函数式编程（What to do），完全支持面向对象程序设计，拥有大量扩展库。
- 胶水语言：可以把多种不同语言编写的程序融合到一起实现无缝拼接，更好地发挥不同语言和工具的优势，满足不同应用领域的需求。

1.1 Python语言简介

- 问题解决：把列表中的所有数字都加5，得到新列表。（命令式编程）

```
>>> x = list(range(10))
```

创建列表

```
>>> x
```

空列表

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
>>> y = []
```

```
>>> for num in x:
```

循环，遍历x中的每个元素

```
    y.append(num+5)
```

列表方法，在尾部追加元素

```
>>> y
```

```
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
>>> [num+5 for num in x]
```

列表推导式

```
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

1.1 Python语言简介

- 问题解决：把列表中的所有数字都加5，得到新列表。（函数式编程）

```
>>> x = list(range(10))
```

```
>>> x
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

定义函数，接收一个数字，加5后返回

```
>>> def add5(num):
```

```
    return num+5
```

把函数add5映射到x中的每个元素

```
>>> list(map(add5, x))
```

```
[5, 6, 7, 8, 9, 10, 11, 12, 13, 14]
```

```
>>> list(map(lambda num: num+5, x))
```

```
[5, 6, 7, 8, 9, 10, 11, 12]
```

lambda表达式，等价于函数add5

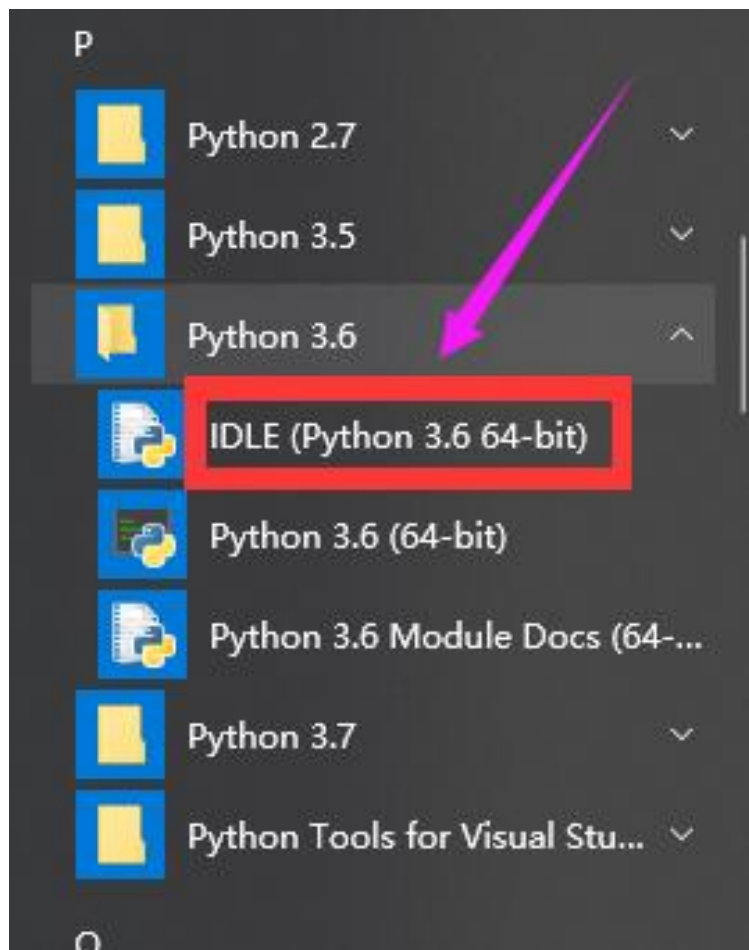
1.2 Python版本简介

- Python目前存在2.x和3.x两个系列的版本，互相之间**不兼容**。
- 在选择Python版本的时候，一定要先考虑清楚自己学习Python的目的是什么，打算做哪方面的开发，该领域或方向有哪些扩展库可用，这些扩展库最高支持哪个版本的Python。这些问题全部确定以后，再最终确定选择哪个版本。
- Python 2.x系列已于2020年1月1日**全面放弃维护和更新**。

1.3 Python开发环境安装与配置

- 默认编程环境：**IDLE**：Python内置IDE，随Python安装包提供
- 其他常用开发环境：
 - **PyCharm**：由著名的JetBrains公司开发，带有一整套可以帮助用户在使用Python语言开发时提高其效率的工具，比如调试、语法高亮、Project管理、代码跳转、智能提示、自动完成、单元测试、版本控制
 - **Anaconda3（内含Jupyter和Spyder）**：<https://www.anaconda.com/download>
 - **Anaconda**，中文大蟒蛇，是一个开源的Python发行版本，其包含了conda、Python等180多个科学包及其依赖项。
 - 安装Python或 Anaconda、Pycharm（指定Python解释器，可指定python或anaconda）

1.3.1 IDLE

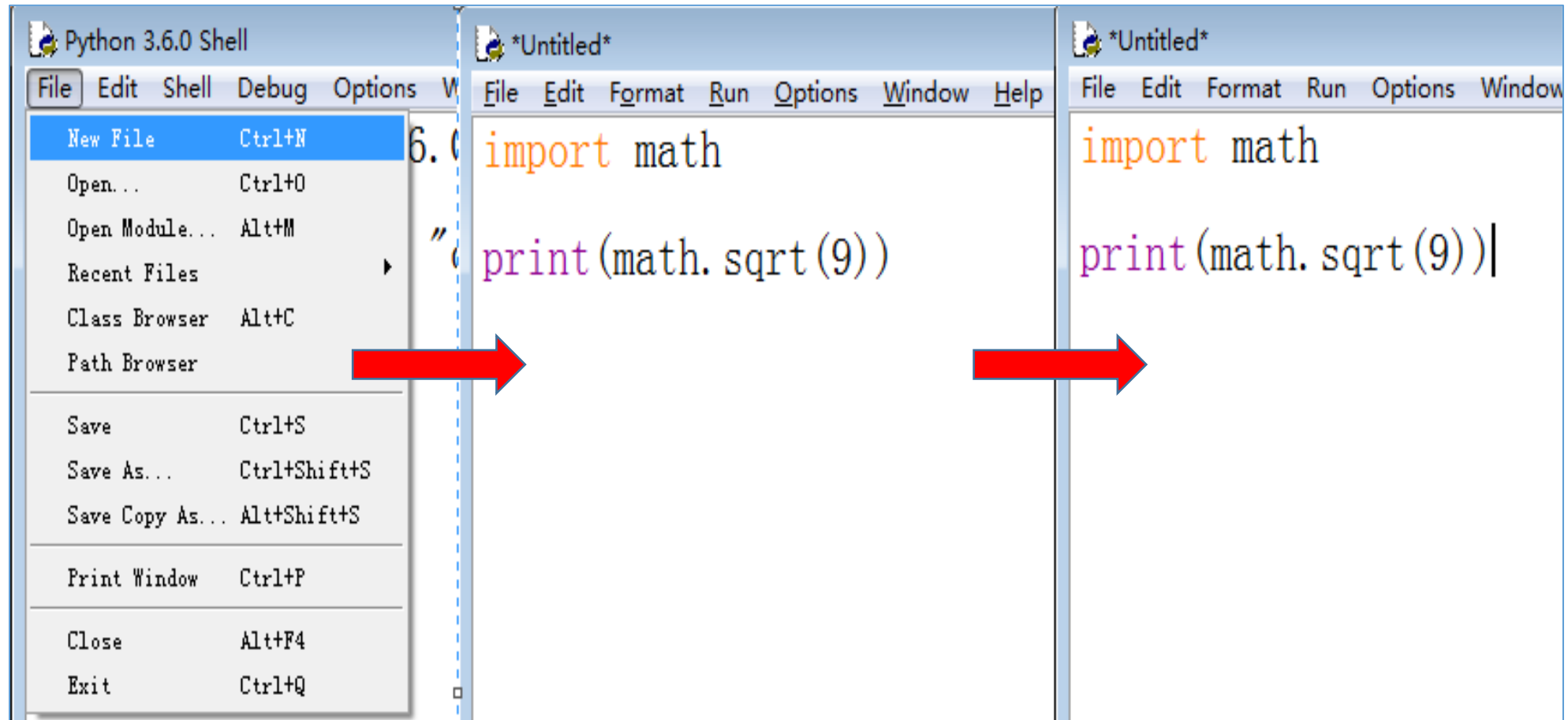


A screenshot of the 'Python 3.6.0 Shell' window. The window has a menu bar with 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The main text area shows the following content:
Python 3.6.0 (v3.6.0:41df79263a11, Dec 23 2016, 08:06:12) [MSC v.1900 64 bit (AMD64)] on win32
Type "copyright", "credits" or "license()" for more information.
>>> print('Hello World!')
Hello World!
>>> import math
>>> print(math.sqrt(9))
3.0
>>>

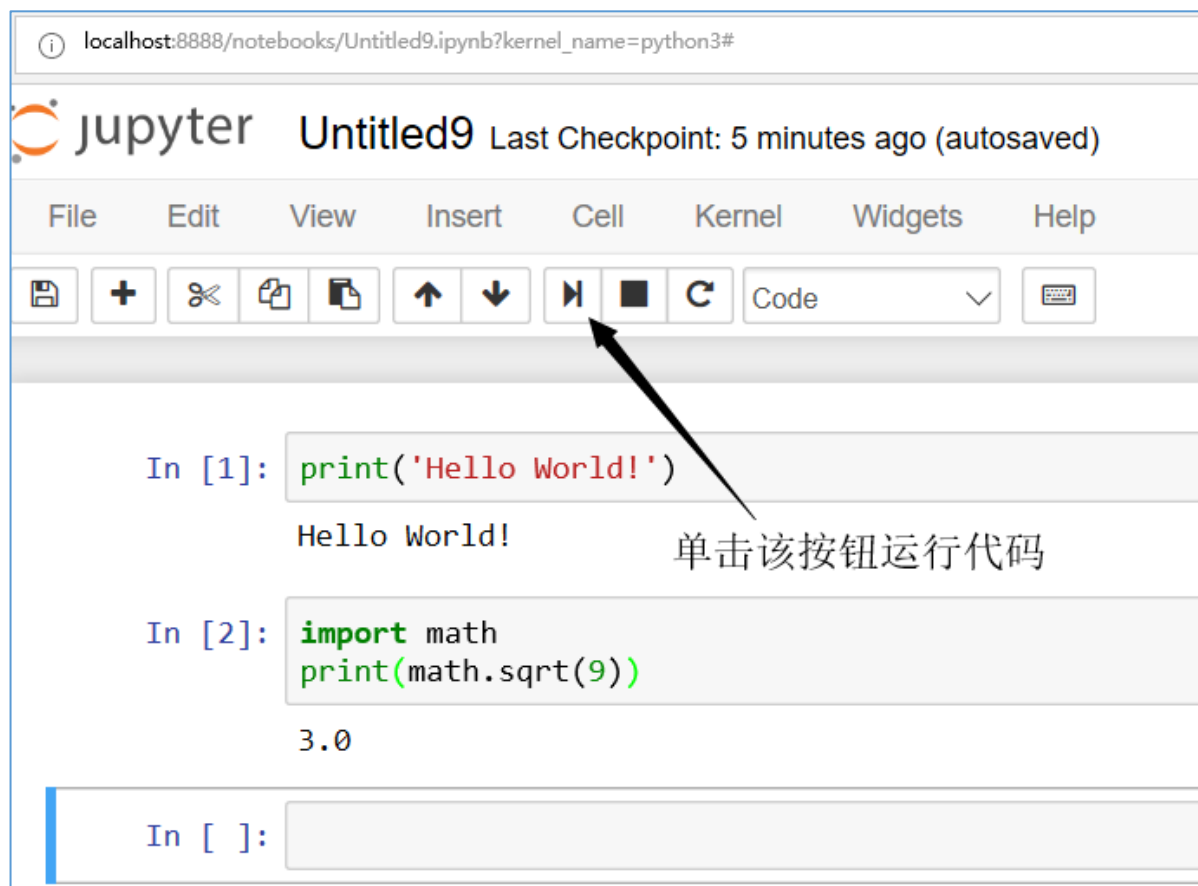
命令提示符

每次只能执行一条语句

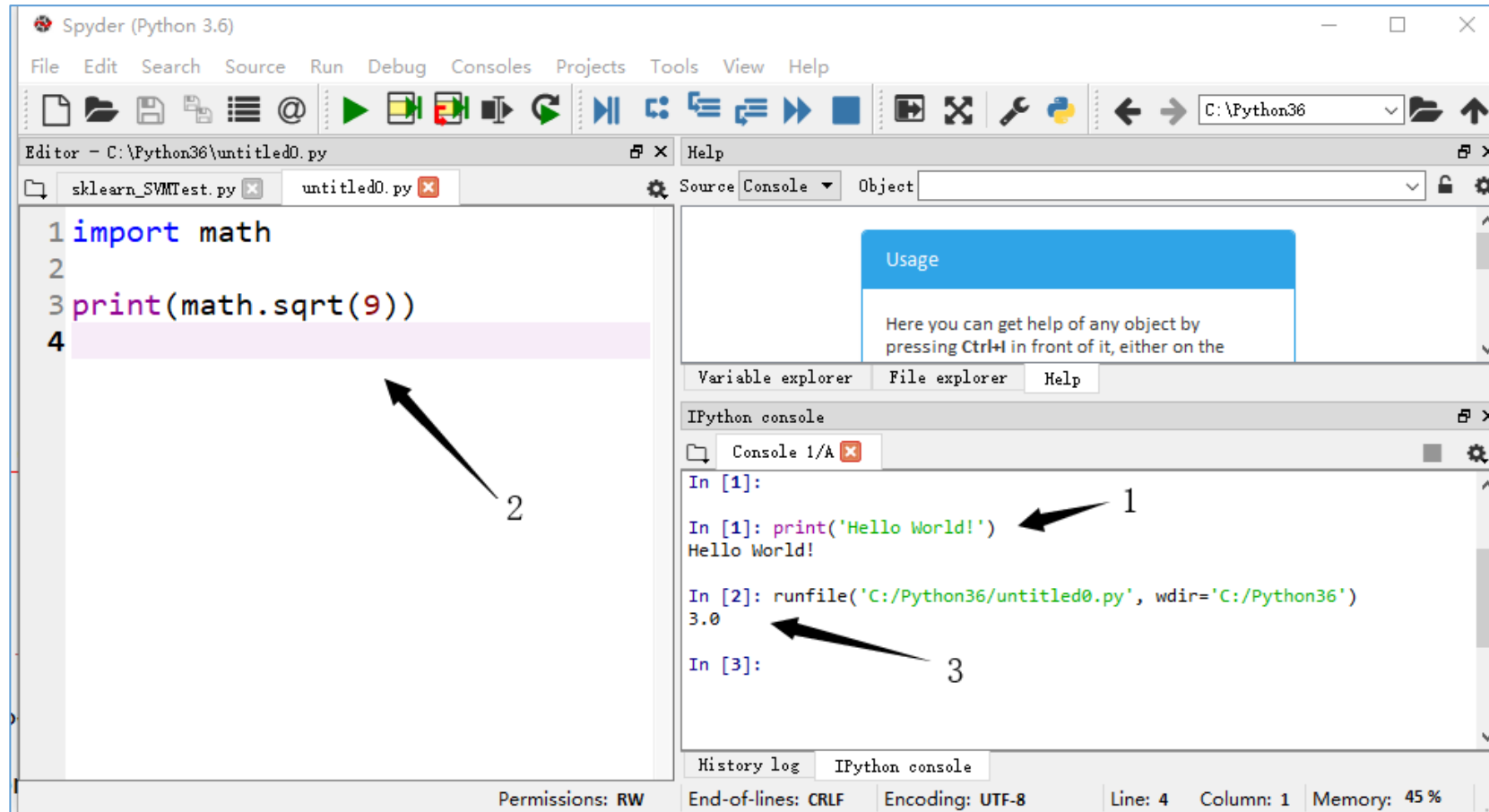
1.3.1 IDLE



1.3.2 Anaconda3



1.3.2 Anaconda3



1.4 Python编程规范与代码优化建议

(1) 缩进

- ✓ 类定义、函数定义、选择结构、循环结构、with块，行尾的冒号表示缩进的开始。
- ✓ python程序是依靠代码块的缩进来体现代码之间的逻辑关系的，缩进结束就表示一个代码块结束了。
- ✓ 同一个级别的代码块的缩进量必须相同。
- ✓ 一般而言，以4个空格为基本缩进单位。

```
with open(fn) as fp:
    for line in csv.reader(fp):
        if line:
            print(*line)
```

1.4 Python编程规范与代码优化建议

(2) 每个import语句只导入一个模块，最好按标准库、扩展库、自定义库的顺序依次导入。

```
import csv
import random
import datetime
import pandas as pd
import matplotlib.pyplot as plt
```

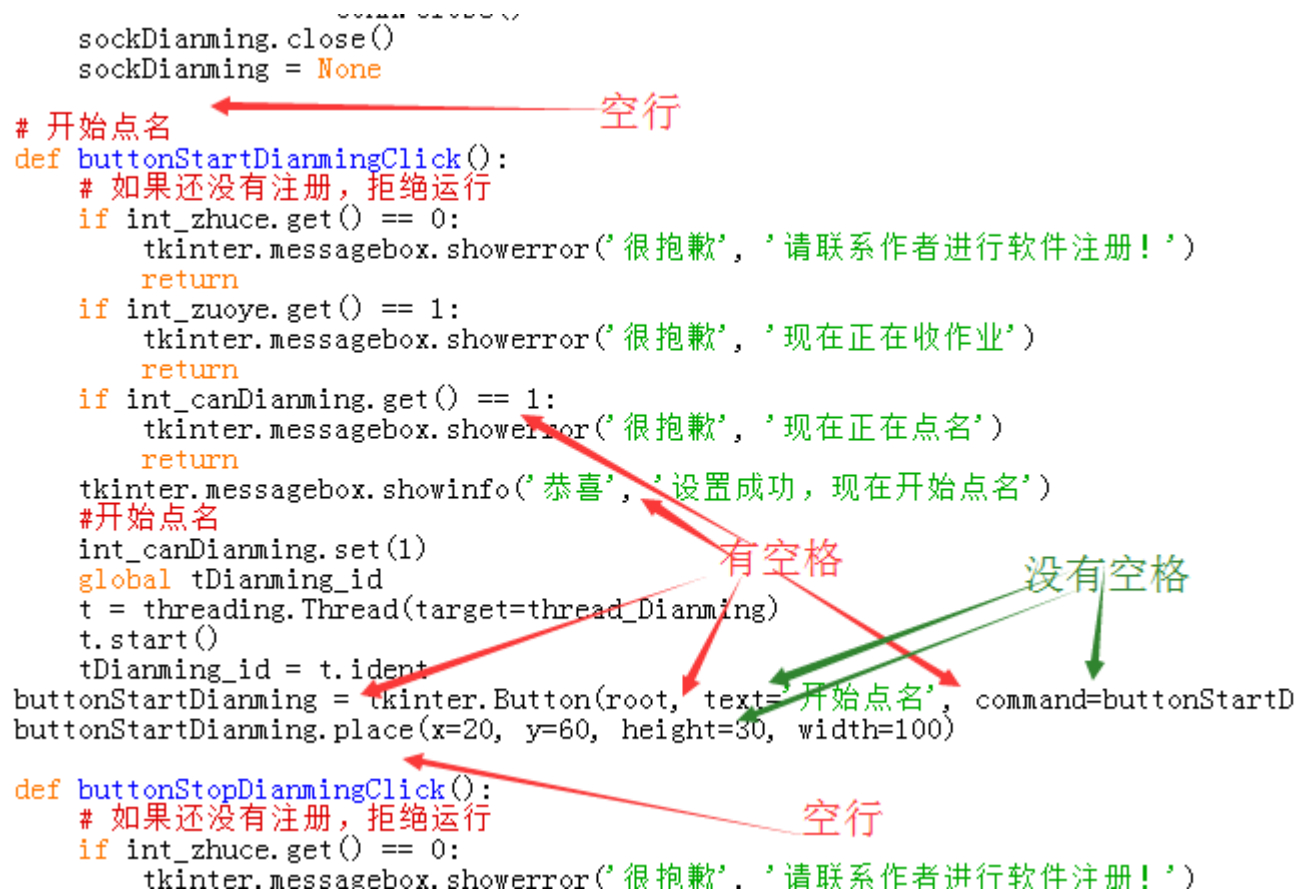
1.4 Python编程规范与代码优化建议

(3) 最好在每个类、函数定义和一段完整的功能代码之后增加一个空行，在运算符两侧各增加一个空格，逗号后面增加一个空格。

```
sockDianming.close()
sockDianming = None

# 开始点名
def buttonStartDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册!')
        return
    if int_zuoye.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在收作业')
        return
    if int_canDianming.get() == 1:
        tkinter.messagebox.showerror('很抱歉', '现在正在点名')
        return
    tkinter.messagebox.showinfo('恭喜', '设置成功，现在开始点名')
    #开始点名
    int_canDianming.set(1)
    global tDianming_id
    t = threading.Thread(target=thread_Dianming)
    t.start()
    tDianming_id = t.ident
    buttonStartDianming = tkinter.Button(root, text='开始点名', command=buttonStartD
    buttonStartDianming.place(x=20, y=60, height=30, width=100)

def buttonStopDianmingClick():
    # 如果还没有注册，拒绝运行
    if int_zhuce.get() == 0:
        tkinter.messagebox.showerror('很抱歉', '请联系作者进行软件注册!')
```



1.4 Python编程规范与代码优化建议

(4) 尽量不要写过长的语句。如果语句过长，可以考虑拆分成多个短一些的语句，以保证代码具有较好的可读性。如果语句确实太长而超过屏幕宽度，最好使用续行符（line continuation character）“\”，或者使用圆括号将多行代码括起来表示是一条语句。

```
x = 1 + 2 + 3\  
    + 4 + 5\  
    + 6  
  
y = (1 + 2 + 3  
     + 4 + 5  
     + 6)
```

1.4 Python编程规范与代码优化建议

(5) 虽然Python运算符有明确的优先级，但对于复杂的表达式建议在适当的位置使用**括号**使得各种运算的隶属关系和顺序更加明确、清晰。

1.4 Python编程规范与代码优化建议

(6) 注释

- ✓ 以符号#开始，表示本行#之后的内容为注释。
- ✓ 包含在一对三引号'''...'''或"""..."""之间且不属于任何语句的内容将被解释器认为是注释。

```
# 读取并显示上面代码生成的csv文件内容
with open(fn) as fp:
    for line in csv.reader(fp):
        if line:
            print(*line)
```

1.5 扩展库安装方法

- pip在线安装（命令提示符环境，建议切换至Python安装目录中的scripts文件夹执行）
- pip离线安装：<https://www.lfd.uci.edu/~gohlke/pythonlibs/>
- exe安装，不是每个扩展库都支持
- conda在线安装
- 如果机器上安装了多个Python开发环境，那么在一个环境下安装的扩展库无法在另一个环境下使用，需要分别安装。

1.5 扩展库安装方法

pip命令示例	说明
<code>pip download SomePackage[==version]</code>	下载扩展库的指定版本，不安装
<code>pip freeze [> requirements.txt]</code>	以requirements的格式列出已安装模块
<code>pip list</code>	列出当前已安装的所有模块
<code>pip install SomePackage[==version]</code>	在线安装SomePackage模块的指定版本
<code>pip install SomePackage.whl</code>	通过whl文件离线安装扩展库
<code>pip install package1 package2 ...</code>	依次（在线）安装package1、package2等扩展模块
<code>pip install -r requirements.txt</code>	安装requirements.txt文件中指定的扩展库
<code>pip install --upgrade SomePackage</code>	升级SomePackage模块
<code>pip uninstall SomePackage[==version]</code>	卸载SomePackage模块的指定版本

把SomePackage替换
为实际要安装或卸载
的扩展库名

<http://www.lfd.uci.edu/~gohlke/pythonlibs/>
下载时选择合适版本，并且不要修改文件名

1.6 标准库与扩展库对象的导入与使用

- Python默认安装仅包含基本或核心模块，启动时也仅加载了基本模块，在需要时再显式地导入和加载标准库和第三方扩展库（需正确安装），这样可以减小程序运行的压力，并且具有很强的可扩展性。
- 从“木桶原理”的角度来看，这样的设计与安全配置时遵循的“最小权限”原则的思想是一致的，也有助于提高系统安全性。

1.6.1 import 模块名 [as 别名]

```
>>> import math
>>> math.sin(0.5)
0.479425538604203
>>> import random
>>> n = random.random()
>>> n = random.randint(1,100)
>>> n = random.randrange(1, 100)
>>> import os.path as path
path
>>> path.isfile(r'C:\windows\notepad.exe')
True
>>> import numpy as np
>>> a = np.array((1,2,3,4))
>>> a
array([1, 2, 3, 4])
>>> print(a)
[1 2 3 4]
```

```
#导入标准库math
#求0.5（单位是弧度）的正弦

#导入标准库random
#获得[0,1) 内的随机小数
#获得[1,100]区间上的随机整数
#返回[1, 100)区间中的随机整数
#导入标准库os.path，并设置别名为

#导入扩展库numpy，并设置别名为np
#通过模块的别名来访问其中的对象
```

1.6.2 from 模块名 import 对象名[as 别名]

```
>>> from math import sin
```

#只导入模块中的指定对象，访问速度略快

```
>>> sin(3)
```

```
0.1411200080598672
```

```
>>> from math import sin as f
```

#给导入的对象起个别名

```
>>> f(3)
```

```
0.1411200080598672
```

```
>>> from os.path import isfile
```

```
>>> isfile(r'C:\windows\notepad.exe')
```

```
True
```

1.6.3 from 模块名 import *

```
>>> from math import *
```

#导入标准库`math`中所有对象

```
>>> sin(3)
```

#求正弦值

```
0.1411200080598672
```

```
>>> gcd(36, 18)
```

#最大公约数

```
18
```

```
>>> pi
```

#常数 π

```
3.141592653589793
```

```
>>> e
```

#常数 e

```
2.718281828459045
```

```
>>> log2(8)
```

#计算以2为底的对数值

```
3.0
```

```
>>> log10(100)
```

#计算以10为底的对数值

```
2.0
```

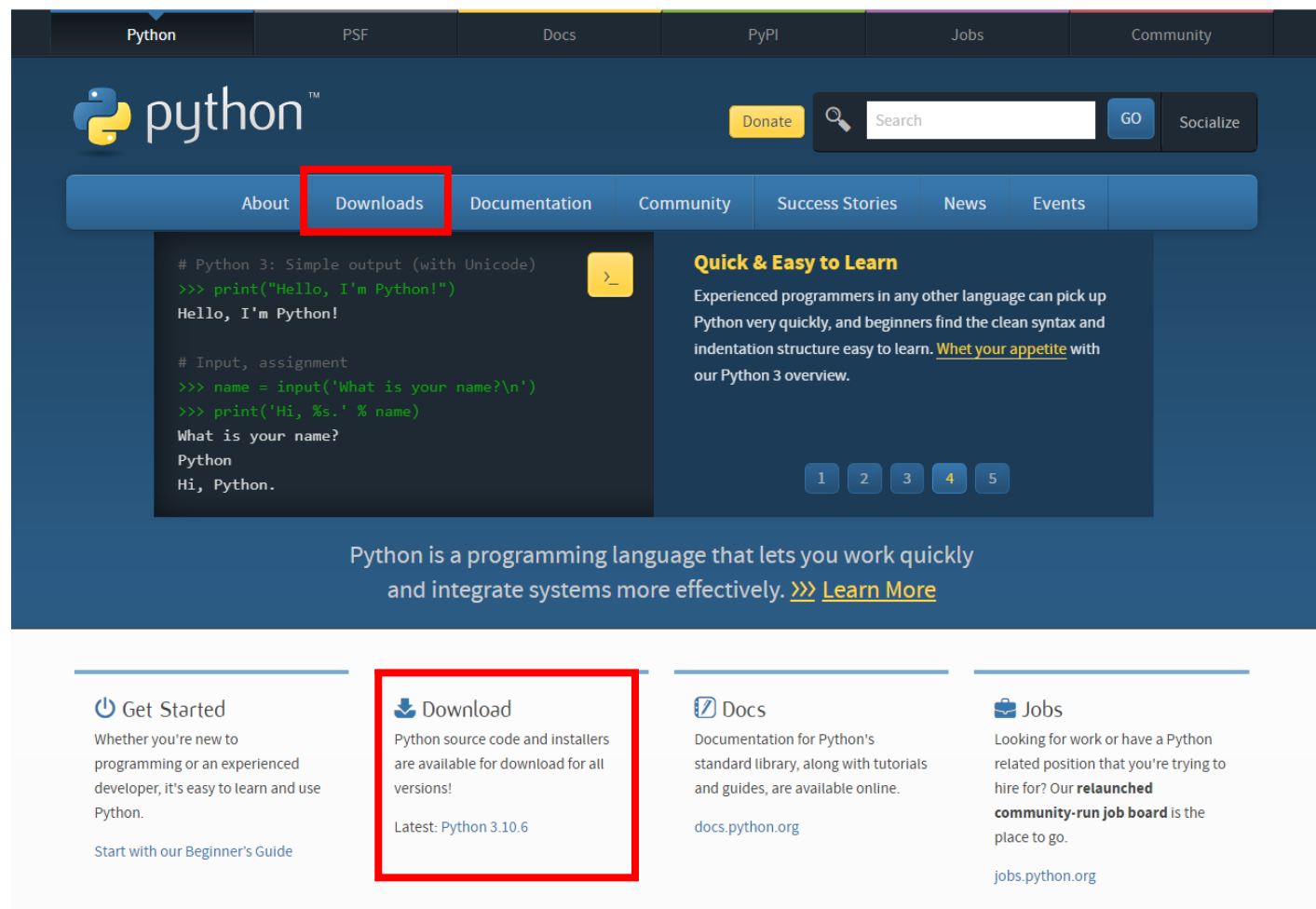
```
>>> radians(180)
```

#把角度转换为弧度

```
3.141592653589793
```

1.7 环境配置

- Python3 下载
- Python3 最新源码，二进制文档，新闻资讯等可以在 Python 的官网查看到：
- Python 官网：<https://www.python.org/>
- 你可以在以下链接中下载 Python 的文档，你可以下载 HTML、PDF 和 PostScript 等格式的文档。
- Python文档下载地址：<https://www.python.org/doc/>



1.7 环境配置



Python 已经被移植在许多平台上（经过改动使它能够工作在不同平台上）。您需要下载适用于您使用平台的二进制代码，然后安装 Python。

如果您平台的二进制代码是不可用的，你需要使用C编译器手动编译源代码。编译的源代码，功能上有更多的选择性，为 Python 安装提供了更多的灵活性。

以下是各个平台安装包的下载地址：

1.7 环境配置

Window 平台安装 Python:

以下为在 Window 平台上安装 Python 的简单步骤。

打开 WEB 浏览器访问 <https://www.python.org/downloads/windows/>，一般就下载 executable installer，x86 表示是 32 位机子的，x86-64 表示 64 位机子的。

Python Releases for Windows

- [Latest Python 3 Release - Python 3.7.0](#)
- [Latest Python 2 Release - Python 2.7.15](#)
- [Python 3.7.0 - 2018-06-27](#)
 - Download [Windows x86 web-based installer](#)
 - Download [Windows x86 executable installer](#)
 - Download [Windows x86 embeddable zip file](#)
 - Download [Windows x86-64 web-based installer](#)
 - Download [Windows x86-64 executable installer](#)
 - Download [Windows x86-64 embeddable zip file](#)
 - Download [Windows help file](#)
- [Python 3.6.6 - 2018-06-27](#)
 - Download [Windows x86 web-based installer](#)

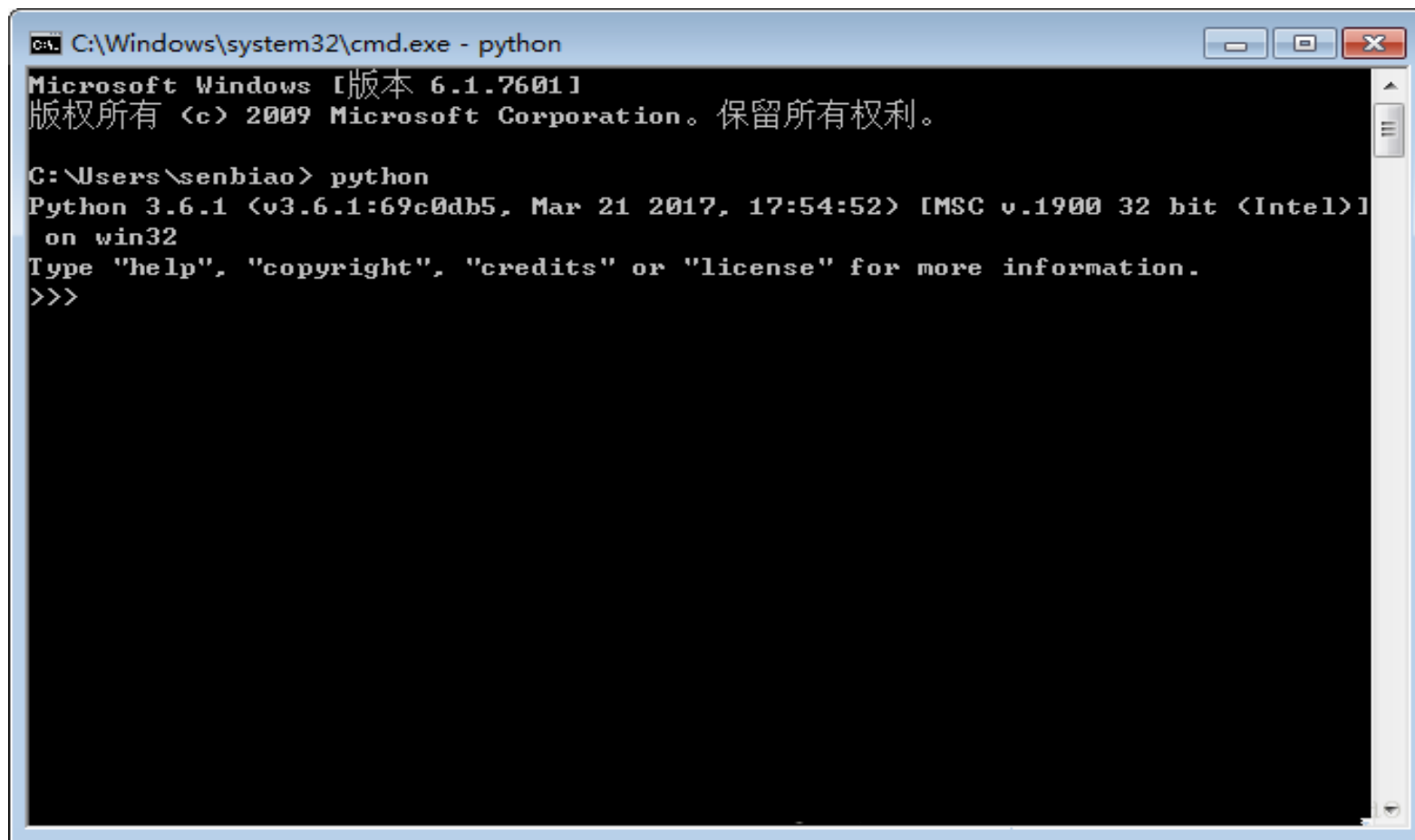
1.7 环境配置



1.7 环境配置

测试是否安装成功:

按 **Win+R** 键, 输入 `cmd` 调出命令提示符, 输入 `python`:



```
C:\Windows\system32\cmd.exe - python
Microsoft Windows [版本 6.1.7601]
版权所有 (c) 2009 Microsoft Corporation。保留所有权利。

C:\Users\senbiao> python
Python 3.6.1 (v3.6.1:69c0db5, Mar 21 2017, 17:54:52) [MSC v.1900 32 bit (Intel)]
on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

1.7 环境配置

也可以在开始菜单中搜索 **IDLE**:



1.7 环境配置

MAC 平台安装 Python:

MAC 系统都自带有 Python2.7 环境，你可以在链接 <https://www.python.org/downloads/mac-osx/> 上下载最新版安装 Python 3.x。

1.7 环境配置

在 **Windows** 设置环境变量

在环境变量中添加Python目录:

在命令提示框中(cmd): 输入

path=%path%;C:\Python 按下"Enter"。

注意: C:\Python 是Python的安装目录。

也可以通过以下方式设置:

右键点击"计算机", 然后点击"属性"

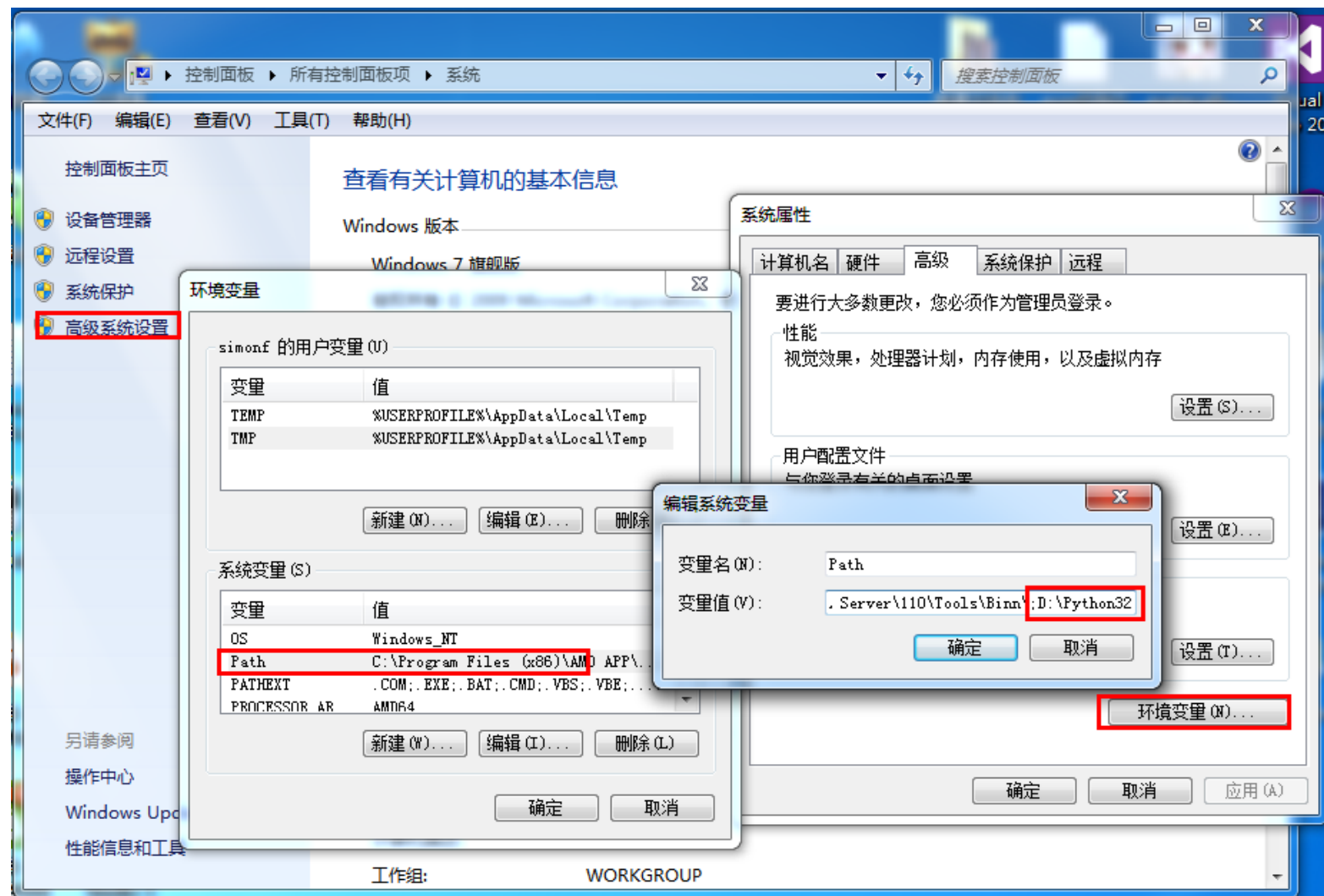
然后点击"高级系统设置"

选择"系统变量"窗口下面的"Path", 双击即可!

然后在"Path"行, 添加python安装路径即可(我的D:\Python32), 所以在后面, 添加该路径即可。 ps: 记住, 路径直接用分号";" 隔开!

最后设置成功以后, 在cmd命令行, 输入命令"python", 就可以有相关显示。

1.7 环境配置



1.7 环境配置

Python 环境变量

下面几个重要的环境变量，它应用于Python：

变量名	描述
PYTHONPATH	PYTHONPATH是Python搜索路径，默认我们import的模块都会从PYTHONPATH里面寻找。
PYTHONSTARTUP	Python启动后，先寻找PYTHONSTARTUP环境变量，然后执行此变量指定的文件中的代码。
PYTHONCASEOK	加入PYTHONCASEOK的环境变量，就会使python导入模块的时候不区分大小写。
PYTHONHOME	另一种模块搜索路径。它通常内嵌于的PYTHONSTARTUP或PYTHONPATH目录中，使得两个模块库更容易切换。

1.7 环境配置

运行 Python

有三种方式可以运行 Python:

1、交互式解释器:

你可以通过命令行窗口进入 Python 并开始在交互式解释器中开始编写 Python 代码。

你可以在 Unix、DOS 或任何其他提供了命令行或者 shell 的系统进行 Python 编码工作。

```
$ python      # Unix/Linux
```

或者

```
C:>python     # Windows/DOS
```

1.7 环境配置

2、命令行脚本

在你的应用程序中通过引入解释器可以在命令行中执行Python脚本，如下所示：

```
$ python script.py      # Unix/Linux
```

或者

```
C:>python script.py      # Windows/DOS
```

1.7 环境配置

3、集成开发环境（IDE：Integrated Development Environment）： PyCharm

PyCharm 是由 JetBrains 打造的一款 Python IDE，支持 macOS、Windows、Linux 系统。

PyCharm 功能：调试、语法高亮、Project 管理、代码跳转、智能提示、自动完成、单元测试、版本控制.....

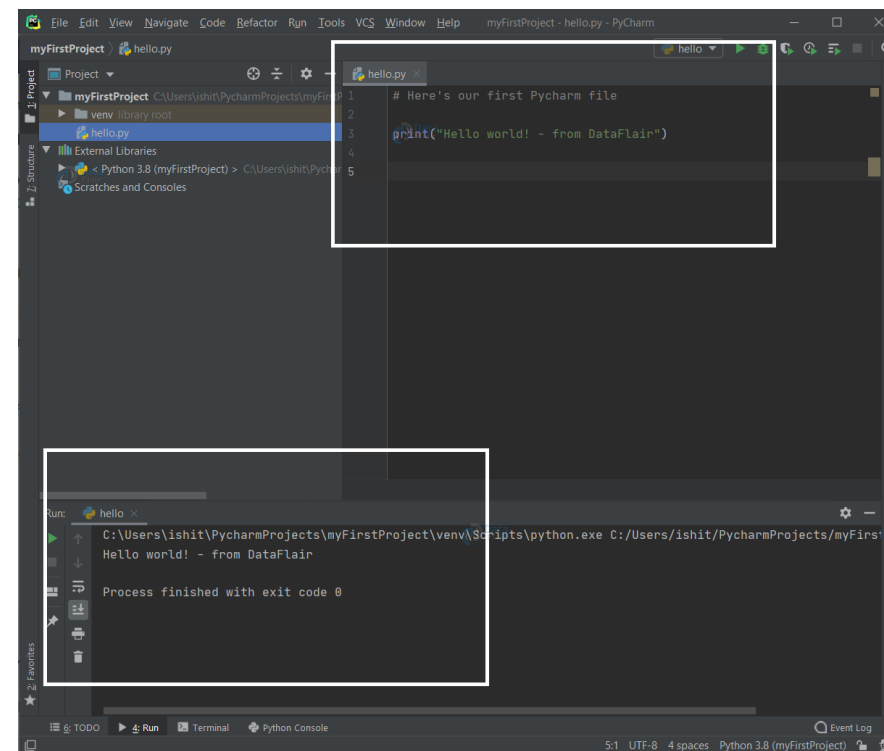
PyCharm 下载地址：

<https://www.jetbrains.com/pycharm/download/>

PyCharm 安装地址：<http://www.runoob.com/w3cnote/pycharm-windows-install.html>

Professional（专业版，收费）：完整的功能，可试用 30 天。

Community（社区版，免费）：阉割版的专业版。



1.8 在线教程

RUNOOB.COM

<https://www.runoob.com/>

RUNOOB.COM

numpy

首页 HTML CSS JAVASCRIPT VUE BOOTSTRAP NODEJS PYTHON3 PYTHON2 JAVA C C++ C# GO SQL LINUX

Python 基础教程

Python 基础教程

Python 简介

Python 环境搭建

Python 中文编码

Python 基础语法

Python 变量类型

Python 运算符

Python 条件语句

Python 循环语句

Python While 循环语句

Python for 循环语句

Python 循环嵌套

Python break 语句

Python continue 语句

Python pass 语句

Python Number(数字)

Python 字符串

Python 列表(List)

Python 元组

Python 字典(Dictionary)

Python 循环语句

Python for 循环语句

Python While 循环语句

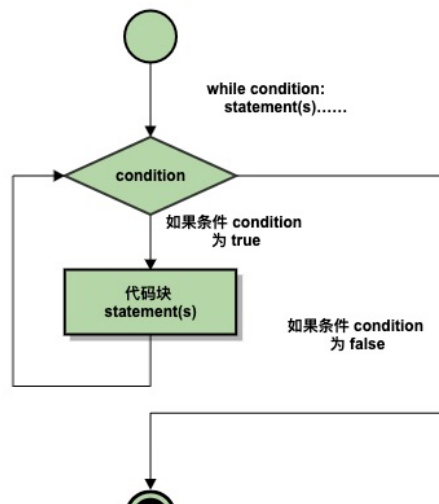
Python 编程中 while 语句用于循环执行程序，即在某条件下，循环执行某段程序，以处理需要重复处理的相同任务。其基本形式为：

```
while 判断条件(condition):  
    执行语句(statement(s)).....
```

执行语句可以是单个语句或语句块。判断条件可以是任何表达式，任何非零、或非空 (null) 的值均为true。

当判断条件假 false 时，循环结束。

执行流程图如下：



1.8 在线教程



<https://leetcode.cn/problemset/all/>

数组 1352 字符串 635 哈希表 481 动态规划 462 数学 430 深度优先搜索 309 排序 304 贪心 266
广度优先搜索 245 树 240 二分查找 220 数据库 210 矩阵 206 二叉树 205 双指针 188 位运算 173 栈 159
设计 145 堆（优先队列） 136 图 127 回溯 117 模拟 113 前缀和 104 链表 101 计数 96 滑动窗口 84
并查集 76 递归 62 二叉搜索树 55 单调栈 54 分治 54 有序集合 53 字典树 51 枚举 42 队列 42
状态压缩 40 记忆化搜索 39 几何 37 拓扑排序 36 线段树 33 哈希函数 26 博弈 24 树状数组 24
数据流 23 字符串匹配 20 最短路径 19 滚动哈希 18 数论 18 交互 18 组合数学 18 随机化 14 归并排序 12
双向链表 12 单调队列 11 快速选择 11 迭代器 10 脑筋急转弯 10 概率与统计 9 多线程 9 桶排序 8
后缀数组 6 计数排序 6 最小生成树 5 扫描线 4 Shell 4 水塘抽样 4 欧拉回路 3 基数排序 3
强连通分量 2 双连通分量 2 拒绝采样 2 收起

数组 1352 字符串 635 哈希表 481 动态规划 462 数学 430 深度优先搜索 309 排序 304 贪心 266 广度展开

全部题目

算法

数据库

Shell

多线程

题单

难度

状态

标签

搜索题目、编号或内容

随机一题

状态	题目	题解	通过率	难度	出现频率
🔒	1475. 商品折扣后的最终价格	537	73.3%	简单	
	1. 两数之和	18993	52.8%	简单	
	2. 两数相加	10391	42.1%	中等	
	3. 无重复字符的最长子串	11557	39.0%	中等	
	4. 寻找两个正序数组的中位数	5795	41.6%	困难	
	5. 最长回文子串	6331	37.1%	中等	
	6. Z 字形变换	3928	52.0%	中等	
	7. 整数反转	6577	35.4%	中等	
	8. 字符串转换整数 (atoi)	3737	21.5%	中等	
	9. 回文数	7077	57.0%	简单	
	10. 正则表达式匹配	1832	31.7%	困难	
	11. 盛最多水的容器	4598	61.1%	中等	
	12. 整数转罗马数字	2534	66.2%	中等	
	13. 罗马数字转整数	5621	62.4%	简单	
	14. 最长公共前缀	5699	42.9%	简单	



中国银联专场竞赛
2023 届校园招聘专场
09.16 周五下午 17:30 开赛

miHoYo
第 309 场力扣周赛
米哈游专场
09.04 周日上午 10:30 开赛

千挂科技
第 86 场力扣双周赛
千挂科技专场
09.03 周六晚 22:30 开赛

精选题单

- HOT 🔥 LeetCode 热题 HOT 100
- SQL 💙 LeetCode 精选数据库 70 题
- 200 💖 LeetCode 精选算法 200 题
- 🏆 力扣杯 - 竞赛合集
- 50 👤 腾讯精选练习 50 题
- TOP 📖 LeetCode 精选 TOP 面试题

1.8 在线教程

力扣 学习 题库 讨论 竞赛 求职 商店

题目描述

评论 (11.8k)

题解 (19.0k)

提交记录

1. 两数之和

难度 简单

15219

☆

题解新功能上线啦! 你可以在这里撰写你的解题思路与大家一同交流分享。

接收动态

反馈

给定一个整数数组 `nums` 和一个目标值 `target`，请你在该数组中找出和为目标值 `target` 的那两个整数，并返回它们的数组下标。

你可以假设每种输入只会对应一个答案。但是，数组中同一个元素在答案里不能重复出现。

你可以按任意顺序返回答案。

示例 1:

输入: `nums = [2,7,11,15], target = 9`
输出: `[0,1]`
解释: 因为 `nums[0] + nums[1] == 9`，返回 `[0, 1]`。

示例 2:

输入: `nums = [3,2,4], target = 6`
输出: `[1,2]`

示例 3:

输入: `nums = [3,3], target = 6`
输出: `[0,1]`

C++

智能模式

C++

Java

Python

Python3

C

C#

JavaScript

```
tion {  
  
<int> twoSum(vector<int>& nums, int target) {
```